Number Theory Study Guide

Factors

A factor of a number is an integer that divides the number completely without leaving a remainder. For example, the factors of 12 are: 1, 2, 3, 4, 6, and 12.

Finding All Factors

```java
public class Factors {
    public static void findFactors(int n) {
        System.out.print("Factors of " + n + ": ");
        for (int i = 1; i <= n; i++) {
            if (n % i == 0) {
                System.out.print(i + " ");
            }
        }
        System.out.println();
    }

    public static void main(String[] args) {
        findFactors(12);
        findFactors(20);
    }
}
```

Optimized Factor Finding

Instead of checking all numbers up to n, we only need to check up to √n:

```java
public class OptimizedFactors {
    public static void findFactors(int n) {
        System.out.print("Factors of " + n + ": ");
        for (int i = 1; i * i <= n; i++) {
            if (n % i == 0) {
                System.out.print(i + " ");
                if (i != n / i) {
                    System.out.print(n / i + " ");
                }
            }
        }
        System.out.println();
    }
}
```

Prime Numbers

A prime number is a natural number greater than 1 that has no positive divisors other than 1 and itself. Examples: 2, 3, 5, 7, 11, 13, 17, 19, 23.

Checking if a Number is Prime

```java
public class PrimeCheck {
    public static boolean isPrime(int n) {
        if (n <= 1) return false;
        if (n <= 3) return true;
        if (n % 2 == 0 || n % 3 == 0) return false;

        for (int i = 5; i * i <= n; i += 6) {
            if (n % i == 0 || n % (i + 2) == 0) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        int[] numbers = {2, 17, 25, 29, 35};
        for (int num : numbers) {
            System.out.println(num + " is prime: " + isPrime(num));
        }
    }
}
```

Sieve of Eratosthenes

An efficient algorithm to find all prime numbers up to a given limit:

```java
public class SieveOfEratosthenes {
    public static void sieve(int n) {
        boolean[] prime = new boolean[n + 1];
        Arrays.fill(prime, true);
        prime[0] = prime[1] = false;

        for (int p = 2; p * p <= n; p++) {
            if (prime[p]) {
                for (int i = p * p; i <= n; i += p) {
                    prime[i] = false;
                }
            }
        }

        System.out.print("Prime numbers up to " + n + ": ");
        for (int i = 2; i <= n; i++) {
            if (prime[i]) {
                System.out.print(i + " ");
            }
        }
        System.out.println();
    }
}
```

Factorial

The factorial of a non-negative integer n is the product of all positive integers less than or equal to n. It is denoted as n! (n factorial).

Iterative Factorial

```java
public class Factorial {
    public static long factorial(int n) {
        if (n < 0) return -1; // Invalid input

        long result = 1;
        for (int i = 1; i <= n; i++) {
            result *= i;
        }
        return result;
    }

    public static void main(String[] args) {
        for (int i = 0; i <= 10; i++) {
            System.out.println(i + "! = " + factorial(i));
        }
    }
}
```

Recursive Factorial

```java
public class RecursiveFactorial {
    public static long factorial(int n) {
        if (n < 0) return -1; // Invalid input
        if (n == 0 || n == 1) return 1;
        return n * factorial(n - 1);
    }

    public static void main(String[] args) {
        System.out.println("5! = " + factorial(5));
        System.out.println("7! = " + factorial(7));
    }
}
```

Iteration

Iteration is the repetition of a process or set of instructions. In programming, we use loops to perform iteration.

Types of Loops in Java

For Loop

```java
// Print numbers 1 to 10
for (int i = 1; i <= 10; i++) {
    System.out.print(i + " ");
}
```

While Loop

```java
// Calculate sum of first n natural numbers
int n = 10;
int sum = 0;
int i = 1;
while (i <= n) {
    sum += i;
    i++;
}
System.out.println("Sum = " + sum);
```

Do-While Loop

```java
// Input validation example
Scanner scanner = new Scanner(System.in);
int number;
do {
    System.out.print("Enter a positive number: ");
    number = scanner.nextInt();
} while (number <= 0);
```

Nested Loops

```java
// Print multiplication table
for (int i = 1; i <= 10; i++) {
    for (int j = 1; j <= 10; j++) {
        System.out.printf("%4d", i * j);
    }
    System.out.println();
}
```

Asymptotic Analysis

Asymptotic analysis is used to describe the behavior of algorithms as the input size approaches infinity. It helps us understand how the runtime or space requirements of an algorithm grow with input size.

Big O Notation

Big O notation describes the upper bound of an algorithm's time complexity.

Common Time Complexities

- **O(1) - Constant Time**: The algorithm takes the same amount of time regardless of input size
- **O(log n) - Logarithmic Time**: Time increases logarithmically with input size
- **O(n) - Linear Time**: Time increases linearly with input size
- **O(n log n) - Linearithmic Time**: Common in efficient sorting algorithms
- **O(n^2) - Quadratic Time**: Time increases quadratically with input size
- **O(2^n) - Exponential Time**: Time doubles with each additional input element

Examples of Different Time Complexities

O(1) - Constant Time

```java
public int getFirstElement(int[] arr) {
    return arr[0]; // Always takes same time
}
```

O(n) - Linear Time

```java
public int findMax(int[] arr) {
    int max = arr[0];
    for (int i = 1; i < arr.length; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}
```

$O(n^2)$ - Quadratic Time

```java
public void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap elements
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

Time Complexity

Time complexity measures how the execution time of an algorithm changes with the size of the input data.

Calculating Time Complexity

1. **Count the basic operations**: Focus on the most frequently executed operations
2. **Express as a function of input size**: Usually denoted as T(n)
3. **Find the dominant term**: The term that grows fastest as n increases
4. **Drop constants and lower-order terms**: Focus on the growth rate

Examples

Linear Search

```java
public int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == target) {
            return i;
        }
    }
    return -1;
}
// Time Complexity: O(n) - worst case we check all elements
```

Binary Search

```java
public int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
// Time Complexity: O(log n) - we halve the search space each time
```

Best, Average, and Worst Case

- **Best Case**: Minimum time required for any input of size n
- **Average Case**: Average time over all possible inputs of size n
- **Worst Case**: Maximum time required for any input of size n

Space Complexity

Space complexity measures how much extra memory space an algorithm uses relative to the input size.

Types of Space Usage

1. **Input Space**: Memory used to store the input
2. **Auxiliary Space**: Extra memory used by the algorithm (excluding input)
3. **Total Space**: Input space + Auxiliary space

Examples

Constant Space - O(1)

```java
public int sum(int a, int b) {
    return a + b; // Uses only a fixed amount of extra space
}
```

Linear Space - O(n)

```java
public int[] createArray(int n) {
    int[] arr = new int[n]; // Space grows linearly with n
    for (int i = 0; i < n; i++) {
        arr[i] = i * i;
    }
    return arr;
}
```

Recursive Space Complexity

```java
public int fibonacci(int n) {
    if (n <= 1) return n;
    return fibonacci(n - 1) + fibonacci(n - 2);
}
// Space Complexity: O(n) due to recursive call stack
```

Space-Time Tradeoffs

Sometimes we can reduce time complexity by using more space, or vice versa.

Example: Fibonacci with Memoization

```java
public class FibonacciMemo {
    private static Map<Integer, Long> memo = new HashMap<>();

    public static long fibonacci(int n) {
        if (n <= 1) return n;

        if (memo.containsKey(n)) {
            return memo.get(n);
        }

        long result = fibonacci(n - 1) + fibonacci(n - 2);
        memo.put(n, result);
        return result;
    }
}
// Time: O(n), Space: O(n) - trades space for better time complexity
```

Quiz Questions

1. What is the time complexity of finding all factors of a number n using the basic approach? a) O(1) b) O(log n) c) O(n) d) $O(n^2)$

Answer: c) O(n)

2. Which of the following is NOT a prime number? a) 17 b) 19 c) 21 d) 23

Answer: c) 21

3. What is the value of 5! (5 factorial)? a) 25 b) 60 c) 120 d) 150

Answer: c) 120

4. What is the space complexity of the recursive factorial function? a) O(1) b) O(log n) c) O(n) d) $O(n^2)$

Answer: c) O(n)

5. Which time complexity represents the most efficient algorithm for large inputs? a) $O(n^2)$ b) O(n log n) c) O(n) d) $O(2^n)$

Answer: c) O(n)

Practice Coding Questions

1. Write a Java method that returns the count of prime numbers between 1 and n (inclusive).

2. Implement a function that finds the Greatest Common Divisor (GCD) of two numbers using the Euclidean algorithm.

3. Create a method that checks if a given number is a perfect number (a number that equals the sum of its proper positive divisors).

4. Write a program that generates the first n terms of the Fibonacci sequence and analyze its time and space complexity.

5. Implement a function that finds all prime factors of a given number and returns them in an array.